

MASTER THE ORDER: STACKS & QUEUES IN ACTION!

Comprehensive Computer Science Lesson Guide with Step-by-Step Code Examples

1. Introduction to Data Structures

In computer science, linear data structures organize data elements sequentially. The order in which elements are added and retrieved determines how the data structure functions. Two fundamental linear data structures are **Stacks** and **Queues**.

2. Understanding Stacks (LIFO)

Core Principle: Last In, First Out (LIFO)

A Stack is a linear data structure where the last element added is the very first element to be removed.

Real-World Analogy: Stack of Hot Idlis or Bangles

Imagine a stack of hot idlis on a plate or bangles placed on a holder. When you want to take an idli, you pick the one off the top. The last idli placed on the stack is the first one you take. Similarly, the last bangle put onto a bangle holder is the first one removed.

Operation	Action	Description
Push	Add Element	Inserts a new element on top of the stack. The newest item becomes the new top.
Pop	Remove Element	Removes and returns the top element from the stack.
Peek	Inspect Top	Views the element currently at the top without removing it.

Code Example 1: Stack Implementation in Python (Using List)

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)
        print(f"PUSH: Added '{item}' | Stack: {self.stack}")

    def pop(self):
        if not self.is_empty():
            removed = self.stack.pop()
            print(f"POP: Removed '{removed}' | Stack: {self.stack}")
            return removed
        print("Error: Stack Underflow")

    def peek(self):
        return self.stack[-1] if not self.is_empty() else None

    def is_empty(self):
        return len(self.stack) == 0

# Execution Trace
s = Stack()
s.push("Page 1")
s.push("Page 2")
s.push("Page 3")
s.pop() # Removes Page 3 (LIFO)
print(f"Current Top: {s.peek()}")
```

Code Example 2: Real-World Stack Application — Text Editor "Undo" Feature

```
class TextEditor:
    def __init__(self):
        self.history = [] # Stack storing past states

    def type_text(self, action):
        self.history.append(action)
        print(f"Action Performed: '{action}'")

    def undo(self):
        if self.history:
            last_action = self.history.pop() # Pop most recent action
            print(f"UNDO: Reverted '{last_action}'")
        else:
            print("Nothing to undo!")

editor = TextEditor()
editor.type_text("Type 'Hello'")
editor.type_text("Change Color to Red")
editor.type_text("Delete Word")
editor.undo() # Undoes 'Delete Word' first
editor.undo() # Undoes 'Change Color to Red'
```

3. Understanding Queues (FIFO)

Core Principle: First In, First Out (FIFO)

A Queue is a linear data structure where the first element added is always the first element to be removed.

Real-World Analogy: Temple Line or Ticket Counter

Think of children standing in a neat line to receive prasad at a temple or passengers waiting for a metro ticket. The person who joins the line first gets served first.

Operation	Action	Description
Enqueue	Join Queue	Adds an element to the rear/end of the queue.
Dequeue	Leave Queue	Removes an element from the front of the queue.
Front / Rear	Inspect Ends	Tracks the first (front) and last (rear) items in the queue.

Code Example 3: Queue Implementation using collections.deque

```
from collections import deque

class Queue:
    def __init__(self):
        self.queue = deque() # O(1) time complexity for popping from front

    def enqueue(self, item):
        self.queue.append(item)
        print(f"ENQUEUE: '{item}' joined queue | Queue: {list(self.queue)}")

    def dequeue(self):
        if self.queue:
            removed = self.queue.popleft() # Removes from FRONT (FIFO)
            print(f"DEQUEUE: Served '{removed}' | Queue: {list(self.queue)}")
            return removed
        print("Queue is empty!")

# Execution Trace
q = Queue()
q.enqueue("Customer A")
q.enqueue("Customer B")
q.enqueue("Customer C")
q.dequeue() # Customer A served first
```

Code Example 4: Real-World Queue Application — Printer Job Queue System

```
from collections import deque

class PrinterQueue:
    def __init__(self):
        self.jobs = deque()

    def add_print_job(self, doc_name, user):
        self.jobs.append((doc_name, user))
        print(f"Job Added: '{doc_name}' from {user}")

    def process_next_job(self):
        if self.jobs:
            doc_name, user = self.jobs.popleft()
            print(f"PRINTING: '{doc_name}' sent by {user} (FIFO Processed)")
        else:
            print("No pending jobs.")

printer = PrinterQueue()
printer.add_print_job("Report.pdf", "Student A (10:00 AM)")
printer.add_print_job("Assignment.docx", "Student B (10:01 AM)")
printer.process_next_job() # Student A's doc prints first
```

Code Example 5: Stack vs Queue Operations in Java

```
import java.util.Stack;
import java.util.LinkedList;
import java.util.Queue;

public class DataStructureDemo {
    public static void main(String[] args) {
        // Java Stack (LIFO)
        Stack<String> stack = new Stack<>();
        stack.push("First");
        stack.push("Second");
        System.out.println("Stack Pop: " + stack.pop()); // Outputs "Second"

        // Java Queue (FIFO)
        Queue<String> queue = new LinkedList<>();
        queue.add("First");
        queue.add("Second");
        System.out.println("Queue Poll: " + queue.poll()); // Outputs "First"
    }
}
```

4. Comparison Summary: Stack vs. Queue

Feature	Stack	Queue
Ordering Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insertion Operation	Push (adds to top)	Enqueue (adds to rear)
Deletion Operation	Pop (removes from top)	Dequeue (removes from front)
Access Points	Single end (Top)	Two ends (Front and Rear)
Common Examples	Undo operations, Bangles on holder	Printer jobs, Metro ticket line

5. Student Practice Questions

Question 1 (Multiple Choice)

Which data structure allows you to undo your most recent move in a software application?

- A) Stack B) Queue C) Array D) Tree

Question 2 (Multiple Choice)

Which real-world scenario directly models a Queue (FIFO) structure?

- A) Bangles stacked on a bangle holder
B) A stack of hot idlis on a plate
C) Waiting in line for a metro ticket or prasad
D) Undoing drawing actions in a paint tool

Question 3 (Multiple Choice)

What is the primary function of the "Pop" operation in a stack?

- A) Add a new element to the top
- B) Remove the top element
- C) View the top element without removing it
- D) Clear the entire stack

Question 4 (Short Answer)

Explain the key difference between LIFO (Last In, First Out) and FIFO (First In, First Out) using an example of your own choice for each.

Question 5 (Scenario Analysis)

Consider a shared computer printer connected to three computers in a school computer lab. If Student A, Student B, and Student C send their printing requests at 10:00 AM, 10:01 AM, and 10:02 AM respectively:

1. Which data structure is utilized to process these requests?
2. In what exact sequence will the documents be printed?

Question 6 (Matching / Identification)

Classify each of the following operations as either a **Stack Operation** or a **Queue Operation**:

1. Enqueue 2. Push 3. Dequeue 4. Peek 5. Pop